

Documentation TP Exam SecOS (SarkOS)

Killian Marty, Aurélien Pouilles

Introduction

Dans le cadre de ce TP examen, nous avons développé un système d'exploitation multi-tâches préemptif minimaliste nommé **SarkOS**. L'objectif était de permettre l'exécution concurrente de deux tâches utilisateur (Ring 3) avec un mécanisme de communication par mémoire partagée et d'affichage via un appel système.

Utilisation

Le projet se situe dans le répertoire `tp_exam`. Le code source principal a été centralisé dans `tp.c` et `debug.c`.

Pour compiler et lancer l'OS, il suffit d'exécuter la commande suivante depuis le dossier `tp_exam` :

```
make qemu
```

La sortie QEMU affichera les traces de debug du noyau ainsi que l'affichage du compteur par la tâche 2.

Fonctionnalités Implémentées

Le système d'exploitation développé intègre, conformément au cahier des charges :

- **Intégration au Noyau** : Le code et les données des tâches utilisateur sont inclus directement dans le binaire du noyau à la compilation via des sections ELF spécifiques (`.user_text` et `.user_data`).
- **Tâches Utilisateur** : Deux tâches distinctes sont implémentées via les fonctions `user1()` et `user2()`, chacune exécutant une boucle infinie en espace utilisateur Ring 3.
- **Gestion de la Mémoire (Pagination)** :
 - La pagination est activée en modifiant le registre `CR0`.
 - Le noyau et le code des tâches sont configurés en *identity mapping* pour les adresses physiques initiales.
 - Chaque tâche dispose de ses propres structures de pagination (PGD/PTB) définies par `pgd_t1` et `pgd_t2` pour assurer l'isolation.
 - Chaque tâche possède sa propre pile noyau de 4KB (`kstack_t1/2`) et sa propre pile utilisateur de 4KB (`ustack_t1/2`).
- **Mémoire Partagée** :
 - Une page de 4KB est partagée entre les deux processus.

- Elle est située à l'adresse physique `0x800000`.
- Elle est mappée à des adresses virtuelles **différentes** dans l'espace d'adressage de chaque tâche : `0xCAFE0000` pour la tâche 1 et `0xBEEF0000` pour la tâche 2.
- **Communication Inter-Processus :**
 - La tâche 1 écrit et incrémente un compteur (`uint32_t`) dans la zone partagée.
 - La tâche 2 lit ce compteur depuis la zone partagée pour demander son affichage.
- **Appel Système (Syscall) :**
 - Un mécanisme d'appel système est installé sur l'interruption `0x80`.
 - Il expose l'interface utilisateur `void sys_counter(uint32_t *counter)`.
 - Le noyau traite le pointeur (adresse virtuelle Ring 3) passé en argument dans le registre `%ebx` et réalise l'affichage via la fonction `debug()`.
- **Ordonnancement Préemptif :**
 - Le gestionnaire de l'interruption horloge (IRQ0 / IDT 32) assure la commutation de contexte automatique entre la tâche 1 et la tâche 2 via la fonction `schedule()`.
 - L'ISR (*Interrupt Service Routine*) détecte dynamiquement si l'interruption survient en mode noyau ou en mode utilisateur en testant les bits de privilège du segment `CS` sauvegardé sur la pile.
- **Cycle de Vie du Noyau :** Après l'initialisation complète de la mémoire, des descripteurs de tâches et des interruptions, le noyau active les interruptions matérielles via l'instruction `sti` et entre dans une boucle infinie stable.

Choix d'Implémentation

Structure du Code

Nous avons fait le choix d'une structure unie au sein du fichier `tp.c`. Plutôt que de disperser la logique dans de multiples fichiers, nous avons implémenté les différentes étapes (initialisation de la GDT, pagination, interruptions, tâches, ordonnancement) dans des fonctions distinctes et séquentielles appelées depuis la fonction principale `tp()`.

Cela permet :

1. Une lecture linéaire du flux d'initialisation du noyau.
2. Une simplicité de gestion pour un projet de cette taille.
3. De regrouper toute la logique spécifique à l'examen en un seul endroit.

Les fonctions clés sont :

- `init_gdt()` / `set_selectors()` : Configuration de la segmentation.
- `init_pagination()` / `activate_pagination()` : Configuration de la mémoire virtuelle.
- `init_interruption()` : Configuration de l'IDT (Syscall & Timer).
- `init_task()` : Préparation des piles et contextes des tâches.
- `schedule()` : Gestion du round-robin.

Ordonnancement

L'ordonnancement est de type **Round-Robin** simple.

- L'interruption horloge (IRQ0, int 32) déclenche le `schedule()`.
- Le noyau sauvegarde le contexte de la tâche courante (si elle existe) et restaure le contexte de la tâche suivante.
- L'alternance se fait strictement entre Tâche 1 et Tâche 2.

Cartographie Mémoire

Nous avons établi une cartographie mémoire précise pour séparer le noyau et les processus utilisateurs tout en permettant la communication.

Espace Virtuel vs Physique

Zone	Adresse Physique	Adresse Virtuelle	Taille	Droits	Description
Noyau	0x00000000	0x00000000	4 MB	Ring 0, RW	Identity Mapping du code/data noyau et des stack noyau (BSS)
Utilisateur (Code/Data)	0x00400000	0x00400000	4 MB	Ring 3, RW	Identity Mapping pour le code des tâches (<code>.user_text</code>) et piles (<code>.user_data</code>)
Tâche 1 (Partagé)	0x00800000	0xCAFE0000	4 KB	Ring 3, RW	Page partagée vue par T1
Tâche 2 (Partagé)	0x00800000	0xBEEF0000	4 KB	Ring 3, RW	Page partagée vue par T2 (Même frame physique)

Piles

- **Piles Noyau (`kstack_tx`)** : Utilisées lors des interruptions/syscalls. Situées dans l'espace noyau (0-4MB).
- **Piles Utilisateur (`ustack_tx`)** : Utilisées par le code utilisateur. Situées dans l'espace utilisateur (4MB+), grâce à l'attribut de section `.user_data`.

Segmentation (GDT)

La GDT est configurée avec un modèle "Flat Memory Model" :

- `0x08` (1) : Kernel Code (0-4GB, DPL 0)
- `0x10` (2) : Kernel Data (0-4GB, DPL 0)
- `0x18` (3) : User Code (0-4GB, DPL 3)
- `0x20` (4) : User Data (0-4GB, DPL 3)
- `0x28` (5) : TSS (Task State Segment) pour la gestion du changement de pile ring 3 -> ring 0.

Gestion des Interruptions

IDT

Deux entrées principales ont été configurées dans l'IDT :

1. **IRQ0 (Int 32)** : Timer (Horloge).

- DPL 0 (Accessible uniquement par le matériel/noyau, pas d' `int 32` user).
- Rôle : Prémption. Appelle `schedule()` .

2. **Syscall (Int 0x80)** : Appel Système.

- DPL 3 (Accessible par l'utilisateur via `int $0x80`).
- Rôle : Service d'affichage. Affiche la valeur pointée par `%ebx` .

TSS

Un seul TSS est utilisé et mis à jour dynamiquement lors du `schedule()` .

- Champ `esp0` : Mis à jour avec le haut de la pile noyau de la *nouvelle* tâche courante.
- Cela permet au processeur de savoir où empiler le contexte utilisateur (SS, ESP, EFLAGS, CS, EIP) lors d'une interruption survenant en Ring 3.